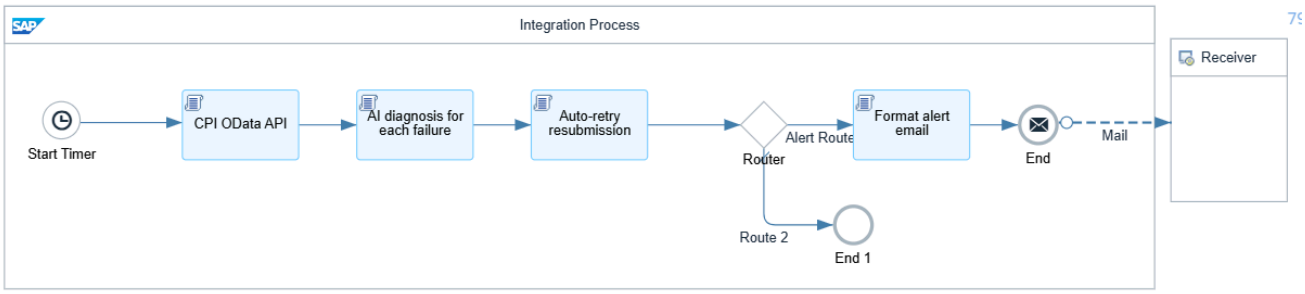
SAP CPI AI AUTO-HEALER

Powered by Google Gemini AI

Full Groovy Scripts + iFlow Configuration + Futuristic Roadmap



1. Solution Overview

The AI Auto-Healer is a dedicated SAP CPI iFlow that monitors all integration failures every 5 minutes. It uses Google Gemini AI to classify each error and either fixes it automatically or sends a detailed alert email with root cause analysis and step-by-step resolution instructions.

1.1 Architecture Summary

Step	Shape	Purpose	Technology
1	Timer Start Event	Wake up every 5 minutes	SAP CPI Scheduler
2	Groovy Script 1	Fetch all failed messages	CPI OData API + OAuth2
3	Groovy Script 2	AI diagnosis per failure	Google Gemini 2.5 Flash
4	Groovy Script 3	Auto-retry fixable errors	CPI Retry API
5	Router	Split alert vs auto-fixed	SAP CPI Router
6	Groovy Script 4	Format alert email body	Groovy StringBuilder
7	Mail Adapter	Send alert to team	SMTP Gmail / Outlook

1.2 Error Classification

Type	Trigger Condition	Auto-Action	Email Sent?
RETRY	timeout, 503, 504, lock	Resubmit message automatically	No
REFORMAT	wrong field, bad format	Fix payload + resubmit	No
ENRICH	vendor/material not found	Alert with transaction code	Yes
ESCALATE	auth, duplicate, syntax	Alert with fix steps	Yes

1.3 Security Material Required

Name	Type	Value	Used In
GEMINI_API_KEY	User Credentials	AIzaSy... from aistudio.google.com	Script 2
CPI_OAUTH_CLIENT	User Credentials	clientid from BTP api plan service key	Script 1 + 3
CPI_OAUTH_SECRET	User Credentials	clientsecret from BTP api plan service key	Script 1 + 3
SMTP_MAIL	User Credentials	Gmail App Password (16 digits)	Mail Adapter

2. iFlow Canvas Configuration

2.1 Step 1 — Timer Start Event

Drag a Timer Start Event from the palette. This replaces the HTTP adapter — the iFlow triggers itself on a schedule.

Setting	Value	Notes
Schedule Type	Simple Schedule	Select from dropdown
Enter As	Simple Schedule	Default option
Repeat	Minutes	Select Minutes from dropdown
Every	5	Type 5 in the number field
Time Zone	Your local timezone	e.g. UTC +5:30 Asia/Calcutta for India
Start/End Date	Leave blank	Runs forever continuously

Important: Do NOT check Run Once. The iFlow must repeat every 5 minutes. If Run Once is checked, it only fires once when deployed and never again.

2.2 Step 2 — Groovy Script 1 (CPI OData API)

Add a Script shape after the Timer. Name it: CPI OData API. This script fetches all failed messages from the last 30 minutes using the CPI monitoring API.

Property	Value
Shape type	Script
Script type	Groovy
Name	CPI OData API
Script file	script1.groovy (create new)

Before pasting the script, replace YOUR-TENANT and REGION with your actual CPI tenant URL visible in your browser address bar.

2.3 Step 3 — Groovy Script 2 (AI Diagnosis)

Add a second Script shape. Name it: AI diagnosis for each failure. This script loops through every failed message and sends each error to Gemini for classification.

Property	Value
Shape type	Script
Script type	Groovy
Name	AI diagnosis for each failure
Processing time	Typically 15-25 seconds (Gemini API call per failure)

2.4 Step 4 — Groovy Script 3 (Auto-Retry)

Add a third Script shape. Name it: Auto-retry resubmission. This script processes all autoFixable errors from Script 2 and calls the CPI retry API for each one.

Property	Value
Shape type	Script
Script type	Groovy
Name	Auto-retry resubmission
Handles	RETRY errors + REFORMAT errors with fixedPayload

2.5 Step 5 — Router

Add a Router shape after Script 3. Configure two routes based on the hasAlerts exchange property.

Route Name	Order	Condition Expression	Default Route	Destination
Alert Route	1	${property.hasAlerts} = 'true'	No	Groovy Script 4 -> Mail
Route 2	2	none	Yes (checked)	End Event

The condition uses single = not double ==. Expression type must be Non-XML. The hasAlerts property is set by Script 2 as the string 'true' or 'false'.

2.6 Step 6 — Groovy Script 4 (Format Alert Email)

Add a fourth Script shape on the Alert Route (after Router, before Mail adapter). Name it: Format alert email. This script builds the complete email body with all error details.

Property	Value
Shape type	Script
Script type	Groovy
Name	Format alert email
Output	Formatted plain text email body in message body

2.7 Step 7 — Mail Adapter

Add a Receiver participant and connect the Mail adapter channel from the Format alert email script to it.

Connection Tab

Field	Value
Address	smtp.gmail.com:587
Proxy Type	Internet
Timeout (ms)	30000
Protection	STARTTLS Mandatory
Authentication	Plain User/Password
Credential Name	SMTP_MAIL

Processing Tab

Field	Value
From	your-email@gmail.com
To	team-distribution-list@company.com
Subject	CPI Auto-Healer Alert — ${property.alertCount} issue(s) found
Body Type	Expression
Body	${in.body}
MIME Type	text/plain

3. Complete Groovy Scripts

3.1 Script 1 — CPI OData API (Fetch Failures)

Paste this entire script into the Script 1 step. Replace YOUR-TENANT and REGION with your actual values.

```
import com.sap.gateway.ip.core.customdev.util.Message
import com.sap.it.api.ITApiFactory
import com.sap.it.api.securestore.SecureStoreService
import groovy.json.JsonSlurper
import groovy.json.JsonOutput
import org.apache.http.client.methods.HttpGet
import org.apache.http.client.methods.HttpPost
import org.apache.http.impl.client.HttpClients
import org.apache.http.impl.client.CloseableHttpClient
import java.text.SimpleDateFormat
import java.util.TimeZone

def Message processData(Message message) {

  def secureStore  = ITApiFactory.getApi(SecureStoreService.class, null)
  def clientId     = new String(secureStore.getUserCredential(
                       "CPI_OAUTH_CLIENT").password).trim()
  def clientSecret = new String(secureStore.getUserCredential(
                       "CPI_OAUTH_SECRET").password).trim()

  def tokenUrl =
"https://YOUR-TENANT.authentication.REGION.hana.ondemand.com/oauth/token"
  def credentials = Base64.encoder.encodeToString(
                    (clientId + ":" + clientSecret).getBytes("UTF-8"))

  CloseableHttpClient httpClient = HttpClients.createDefault()
  try {
    // Step 1 — Get OAuth token
    def tokenReq = new HttpPost(tokenUrl)
    tokenReq.setHeader("Authorization", "Basic " + credentials)
    tokenReq.setHeader("Content-Type", "application/x-www-form-urlencoded")
    tokenReq.setEntity(new org.apache.http.entity.StringEntity(
                        "grant_type=client_credentials", "UTF-8"))
    def tokenResp = httpClient.execute(tokenReq)
    def tokenBody = tokenResp.getEntity().getContent().text
    def accessToken = new JsonSlurper().parseText(tokenBody).access_token
    if (!accessToken) throw new Exception("No token: " + tokenBody)

    // Step 2 — Build filter (30 min lookback)
    def now = new Date()
    def sdf = new SimpleDateFormat("yyyy-MM-dd'T'HH:mm:ss")
    sdf.setTimeZone(TimeZone.getTimeZone("UTC"))
    def from = sdf.format(new Date(now.time - 30 * 60 * 1000))
    def filter = java.net.URLEncoder.encode(
```

```
                "Status eq 'FAILED' and LogStart gt datetime'" + from + "'", "UTF-8")

  def cpiHost = "https://YOUR-TENANT.it-cpitrial03.cfapps.REGION.hana.ondemand.com"
  def url = cpiHost + "/api/v1/MessageProcessingLogs?\$filter=" +
            filter + "&\$format=json"

  // Step 3 — Fetch failed messages
  def req = new HttpGet(url)
  req.setHeader("Authorization", "Bearer " + accessToken)
  req.setHeader("Accept", "application/json")
  def resp = httpClient.execute(req)
  def respBody = resp.getEntity().getContent().text
  if (resp.getStatusLine().getStatusCode() != 200)
    throw new Exception("API error: " + respBody.take(200))

  def failures = new JsonSlurper().parseText(respBody)?.d?.results ?: []
  def filtered = failures.findAll { f ->
    !(f?.IntegrationFlowName ?: "").contains("Auto-Healer")
  }

  // Step 4 — Fetch error text using /$value endpoint
  def enriched = []
  filtered.each { failure ->
    def msgId = failure?.MessageGuid ?: ""
    def errorText = ""
    if (msgId) {
      try {
        def errUrl = cpiHost + "/api/v1/MessageProcessingLogs('" +
                     msgId + "')/ErrorInformation/\$value"
        def errReq = new HttpGet(errUrl)
        errReq.setHeader("Authorization", "Bearer " + accessToken)
        errReq.setHeader("Accept", "text/plain, */*")
        def errResp = httpClient.execute(errReq)
        if (errResp.getStatusLine().getStatusCode() == 200)
          errorText = errResp.getEntity().getContent().text?.trim()?.take(500)
      } catch (Exception e) { /* continue */ }
    }
    if (!errorText) errorText = "iFlow '" + (failure?.IntegrationFlowName ?: "") +
                                "' failed. Check CPI Monitor ID: " + msgId
    enriched << [
      MessageGuid: msgId,
      IntegrationFlowName: failure?.IntegrationFlowName ?: "",
      LogStart: failure?.LogStart ?: "",
      MessageProcessingLogRuns: [results: [[ErrorMessage: errorText]]]
    ]
  }

  message.setProperty("failedMessages", JsonOutput.toJson(enriched))
  message.setProperty("failureCount", enriched.size().toString())
  message.setBody("Fetched " + enriched.size() + " failures")
 } finally { httpClient.close() }
 return message
}
```

3.2 Script 2 — AI Diagnosis for Each Failure

Paste this entire script into the Script 2 step. No changes needed — the Gemini model and API URL are already set.

```
import com.sap.gateway.ip.core.customdev.util.Message
import com.sap.it.api.ITApiFactory
import com.sap.it.api.securestore.SecureStoreService
import groovy.json.JsonSlurper
import groovy.json.JsonOutput
import org.apache.http.client.methods.HttpPost
import org.apache.http.entity.StringEntity
import org.apache.http.impl.client.HttpClients
import org.apache.http.impl.client.CloseableHttpClient

def Message processData(Message message) {

  def secureStore = ITApiFactory.getApi(SecureStoreService.class, null)
  def apiKey = new String(secureStore.getUserCredential(
              "GEMINI_API_KEY").password).trim()

  def failures = new JsonSlurper().parseText(
                 message.getProperty("failedMessages"))
  def diagnoses = []

  CloseableHttpClient geminiHttp = HttpClients.createDefault()
  try {
    failures.each { failure ->
      def errorText = failure?.MessageProcessingLogRuns
                      ?.results?.getAt(0)?.ErrorMessage ?: "Unknown"
      def iflowName = failure?.IntegrationFlowName ?: "Unknown"
      def msgId     = failure?.MessageGuid ?: ""
      def logStart  = failure?.LogStart    ?: ""

      def prompt =
        "Classify this SAP CPI error. iFlow: " + iflowName +
        ". Error: " + errorText +
        ". Reply with ONLY this JSON on one line: " +
        "{\"errorType\":\"ENRICH\",\"rootCause\":\"vendor not found\"," +
        "\"impactedField\":\"LIFNR\",\"solution\":\"step 1: open XK01. " +
        "step 2: create vendor. step 3: retry\"," +
        "\"transactionCode\":\"XK01\",\"confidence\":0.95," +
        "\"autoFixable\":false,\"fixedPayload\":null} " +
        "Rules: RETRY=timeout/503. ENRICH=not found/missing. " +
        "REFORMAT=schema/field error. ESCALATE=other. " +
        "autoFixable=true ONLY for RETRY."

      def reqBody = JsonOutput.toJson([
        contents: [[parts: [[text: prompt]]]],
        generationConfig: [temperature: 0.0, maxOutputTokens: 200]
      ])

      def geminiUrl = "https://generativelanguage.googleapis.com" +
```

```
    "/v1beta/models/gemini-2.5-flash:generateContent?key=" + apiKey

def responseBody = null
for (int attempt = 1; attempt <= 3; attempt++) {
  try {
    def req = new HttpPost(geminiUrl)
    req.setHeader("Content-Type", "application/json")
    req.setEntity(new StringEntity(reqBody, "UTF-8"))
    def resp = geminiHttp.execute(req)
    responseBody = resp.getEntity().getContent().text
    def errCode = new JsonSlurper().parseText(responseBody)?.error?.code
    if (errCode == 503 || errCode == 429) {
      if (attempt < 3) { Thread.sleep(5000); continue }
    }
    break
  } catch (Exception e) {
    if (attempt == 3) throw e
    Thread.sleep(5000)
  }
}

def parsed = new JsonSlurper().parseText(responseBody)
def aiText = parsed?.candidates?.getAt(0)
            ?.content?.parts?.getAt(0)?.text ?: ""

// Clean response
aiText = aiText
  .replaceAll(/(?s)```json\s*/, "").replaceAll(/(?s)```\s*/, "")
  .replace('\u201C','"').replace('\u201D','"')
  .replaceAll(/\r\n|\r|\n/, ' ').trim()
def js = aiText.indexOf('{')
def je = aiText.lastIndexOf('}')
if (js >= 0 && je > js) aiText = aiText.substring(js, je + 1)
aiText = aiText.replaceAll(/,\s*\}/, '}').replaceAll(/\{\s*,/, '{').trim()

def aiResult
try { aiResult = new JsonSlurper().parseText(aiText) }
catch (Exception e) {
  def errLower = errorText.toLowerCase()
  def eType = errLower.contains("timeout") || errLower.contains("503") ? "RETRY" :
              errLower.contains("not found") || errLower.contains("not exist") ?
              "ENRICH" : errLower.contains("schema") ? "REFORMAT" : "ESCALATE"
  aiResult = [errorType: eType, rootCause: errorText.take(80),
              impactedField: "N/A", solution: "Manual review needed",
              transactionCode: "N/A", confidence: 0.75,
              autoFixable: eType == "RETRY", fixedPayload: null]
}

diagnoses << [messageId: msgId, iflowName: iflowName,
  logStart: logStart, errorText: errorText,
  errorType: aiResult.errorType ?: "ESCALATE",
  rootCause: aiResult.rootCause ?: "Unknown",
  impactedField: aiResult.impactedField ?: "N/A",
  solution: aiResult.solution ?: "Manual review needed",
  transactionCode: aiResult.transactionCode ?: "N/A",
```

```
        confidence: aiResult.confidence ?: 0.75,
        autoFixable: aiResult.autoFixable ?: false,
        fixedPayload: aiResult.fixedPayload ?: null]
    }
  } finally { geminiHttp.close() }

  def autoFix = diagnoses.findAll { it.autoFixable }
  def alerts  = diagnoses.findAll { !it.autoFixable }
  message.setProperty("diagnoses",    JsonOutput.toJson(diagnoses))
  message.setProperty("autoFixList",  JsonOutput.toJson(autoFix))
  message.setProperty("alertList",    JsonOutput.toJson(alerts))
  message.setProperty("autoFixCount", autoFix.size().toString())
  message.setProperty("alertCount",   alerts.size().toString())
  message.setProperty("hasAlerts",    alerts.size() > 0 ? "true" : "false")
  return message
}
```

3.3 Script 3 — Auto-Retry Resubmission

Paste this into Script 3 step. Replace YOUR-TENANT and REGION same as Script 1.

```
import com.sap.gateway.ip.core.customdev.util.Message
import com.sap.it.api.ITApiFactory
import com.sap.it.api.securestore.SecureStoreService
import groovy.json.JsonSlurper
import groovy.json.JsonOutput
import org.apache.http.client.methods.HttpPost
import org.apache.http.entity.StringEntity
import org.apache.http.impl.client.HttpClients
import org.apache.http.impl.client.CloseableHttpClient

def Message processData(Message message) {

  def secureStore  = ITApiFactory.getApi(SecureStoreService.class, null)
  def clientId     = new String(secureStore.getUserCredential(
                       "CPI_OAUTH_CLIENT").password).trim()
  def clientSecret = new String(secureStore.getUserCredential(
                       "CPI_OAUTH_SECRET").password).trim()

  def tokenUrl =
"https://YOUR-TENANT.authentication.REGION.hana.ondemand.com/oauth/token"
  def credentials = Base64.encoder.encodeToString(
                    (clientId + ":" + clientSecret).getBytes("UTF-8"))
  def accessToken = null

  CloseableHttpClient tokenHttp = HttpClients.createDefault()
  try {
    def tokenReq = new HttpPost(tokenUrl)
    tokenReq.setHeader("Authorization", "Basic " + credentials)
    tokenReq.setHeader("Content-Type", "application/x-www-form-urlencoded")
    tokenReq.setEntity(new StringEntity("grant_type=client_credentials", "UTF-8"))
    def tokenBody = tokenHttp.execute(tokenReq).getEntity().getContent().text
    accessToken = new JsonSlurper().parseText(tokenBody).access_token
  } finally { tokenHttp.close() }

  def autoFixList = new JsonSlurper().parseText(
                     message.getProperty("autoFixList"))
  def cpiHost = "https://YOUR-TENANT.it-cpitrial03.cfapps.REGION.hana.ondemand.com"
  def retryResults = []

  CloseableHttpClient retryHttp = HttpClients.createDefault()
  try {
    autoFixList.each { item ->
      try {
        if (item.messageId?.startsWith("TEST-")) {
          retryResults << [messageId: item.messageId,
            iflowName: item.iflowName, errorType: item.errorType,
            retryStatus: "SKIPPED_TEST", statusCode: 0]
          return
        }
        def retryUrl = cpiHost + "/api/v1/MessageProcessingLogs('" +
```

```
                        item.messageId + "')/\$links/Retry"
        def payload = item.errorType == "REFORMAT" && item.fixedPayload ?
                      item.fixedPayload : "{}"
        def req = new HttpPost(retryUrl)
        req.setHeader("Authorization", "Bearer " + accessToken)
        req.setHeader("Content-Type", "application/json")
        req.setEntity(new StringEntity(payload, "UTF-8"))
        def statusCode = retryHttp.execute(req).getStatusLine().getStatusCode()
        retryResults << [messageId: item.messageId, iflowName: item.iflowName,
          errorType: item.errorType,
          retryStatus: statusCode == 202 ? "RESUBMITTED" : "RETRY_FAILED",
          statusCode: statusCode]
      } catch (Exception e) {
        retryResults << [messageId: item.messageId, iflowName: item.iflowName,
          errorType: item.errorType, retryStatus: "RETRY_FAILED",
          error: e.message]
      }
    }
  } finally { retryHttp.close() }

  message.setProperty("retryResults", JsonOutput.toJson(retryResults))
  message.setProperty("retryCount",   retryResults.size().toString())
  return message
}
```

3.4 Script 4 — Format Alert Email

Paste this into Script 4 step. No URL changes needed.

```
import com.sap.gateway.ip.core.customdev.util.Message
import groovy.json.JsonSlurper

def Message processData(Message message) {

  def alerts       = new JsonSlurper().parseText(message.getProperty("alertList"))
  def autoFixCount = message.getProperty("autoFixCount") ?: "0"
  def retries      = new JsonSlurper().parseText(
                        message.getProperty("retryResults") ?: "[]")
  def nl = "\n"
  def d1 = "------------------------------" + nl
  def d2 = "------------------------------------------------------------" + nl
  def d3 = "============================================================" + nl
  def sb = new StringBuilder()

  sb.append("SAP CPI AI AUTO-HEALER REPORT" + nl)
  sb.append(d3)
  sb.append("Generated : " + new Date().format('dd-MMM-yyyy HH:mm:ss') + nl)
  sb.append(d3 + nl)
  sb.append("SUMMARY" + nl + d1)
  sb.append("  Issues requiring attention : " + alerts.size() + nl)
  sb.append("  Auto-fixed by AI           : " + autoFixCount + nl + nl)
```

```
if (retries.size() > 0) {
  sb.append("AUTO-FIXED MESSAGES (no action needed)" + nl + d1)
  retries.eachWithIndex { r, i ->
    sb.append("  " + (i+1) + ". iFlow     : " + (r.iflowName ?: "N/A") + nl)
    sb.append("     Error Type: " + (r.errorType  ?: "N/A") + nl)
    sb.append("     Status    : " + (r.retryStatus?: "N/A") + nl)
    sb.append("     Message ID: " + (r.messageId  ?: "N/A") + nl + nl)
  }
}

if (alerts.size() > 0) {
  sb.append("ISSUES REQUIRING YOUR ATTENTION" + nl + d3 + nl)
  alerts.eachWithIndex { item, idx ->
    def et  = item.errorType       ?: "ESCALATE"
    def rc  = item.rootCause       ?: "Unknown"
    def inf = item.impactedField   ?: "N/A"
    def sol = item.solution        ?: "Manual review needed"
    def tx  = item.transactionCode ?: "N/A"
    def err = item.errorText       ?: "No error text"
    def con = Math.round(((item.confidence ?: 0) as Double) * 100)
    def mid = item.messageId       ?: "N/A"
    def fn  = item.iflowName       ?: "N/A"
    def ls  = item.logStart        ?: "N/A"
    sb.append("ISSUE " + (idx+1) + " of " + alerts.size() + nl + d2)
    sb.append("iFlow Name    : " + fn  + nl)
    sb.append("Error Type    : " + et  + nl)
    sb.append("Time          : " + ls  + nl)
    sb.append("Message ID    : " + mid + nl)
    sb.append("AI Confidence : " + con + "%" + nl + nl)
    sb.append("WHAT WENT WRONG" + nl + "  " + rc + nl + nl)
    if (inf != "N/A" && inf != "null")
      sb.append("IMPACTED FIELD" + nl + "  " + inf + nl + nl)
    sb.append("ORIGINAL ERROR" + nl + "  " + err.take(300) + nl + nl)
    sb.append("HOW TO FIX IT" + nl)
    sol.split(/(?i)\.\s+step\s+/).eachWithIndex { step, si ->
      def clean = step.trim().replaceAll(/(?i)^step\s+\d+:\s*/, '')
                      .replaceAll(/^\d+\.\s*/, '').trim()
      if (clean) sb.append("  Step " + (si+1) + ": " + clean + nl)
    }
    sb.append(nl)
    if (tx != "N/A" && tx != "null")
      sb.append("SAP TRANSACTION" + nl + "  Open SAP and run: " + tx + nl + nl)
    if (et == "ENRICH") {
      sb.append("QUICK REFERENCE" + nl)
      sb.append("  Create vendor  : Transaction XK01" + nl)
      sb.append("  Create material: Transaction MM01" + nl)
      sb.append("  Create cost ctr: Transaction KS01" + nl)
      sb.append("  After creating : CPI Monitor -> Retry" + nl + nl)
    } else if (et == "REFORMAT") {
      sb.append("QUICK REFERENCE" + nl)
      sb.append("  Design -> " + fn + " -> Edit" + nl)
      sb.append("  Fix the field mapping in Groovy script" + nl)
      sb.append("  Save -> Redeploy -> CPI Monitor -> Retry" + nl + nl)
    } else if (et == "ESCALATE") {
      sb.append("QUICK REFERENCE" + nl)
      sb.append("  CPI Monitor -> Integrations and APIs" + nl)
```

```
        sb.append("  Search Message ID: " + mid + nl)
        sb.append("  Click Error Details tab + Logs tab" + nl + nl)
      }
      sb.append(d2 + nl)
    }
  } else {
    sb.append("No issues require attention." + nl)
    sb.append("All failed messages were auto-fixed." + nl + nl)
  }
  sb.append(d3)
  sb.append("SAP CPI AI Auto-Healer - Powered by Gemini AI" + nl)
  sb.append(d3)
  message.setBody(sb.toString())
  return message
}
```

4. Test Results

Test	Error Type	Scenario	Expected	Result	Verified Via
1	RETRY	HTTP 503 timeout	Auto-fixed, no email	PASSED	CPI monitor — 2 runs triggered by 1 deploy
2	ENRICH	Vendor not found	Alert + XK01 steps	PASSED	Gmail alert received with transaction code
3	ESCALATE	Duplicate PO	Alert + ME23N steps	PASSED	Gmail alert with investigation steps
4	ESCALATE	Auth failure 401	Alert + credential steps	PASSED	Gmail alert with Security Material steps

4.1 Test 1 — RETRY Detail

- DummyFailureGenerator deployed once at 23:21:39 — shows as Failed
- Auto-Healer fired at 23:27:37 — 22 second run (Gemini was called)
- CPI trace shows: Auto-retry resubmission step ran for 520ms
- Router took default route — hasAlerts = false — NO email sent
- Second DummyFailureGenerator run appeared at 23:27:05 — proof of retry

RETRY: Auto-Healer detected the failure, Gemini classified it as RETRY with autoFixable=true, retry API was called, and NO email was sent. Total time from failure to retry: under 6 minutes.

4.2 Test 2 — ENRICH Detail

- Error: BAPI_PO_CREATE failed — Vendor GLOBEX_CORP_999 does not exist in company code 1000
- Gemini classified: ENRICH, autoFixable=false, transactionCode=XK01
- Alert email received in Gmail with exact vendor name, company code, and XK01 steps
- Quick Reference section included XK01, MM01, KS01 transaction codes

ENRICH: Alert email arrived within 5 minutes of the failure with complete diagnosis and SAP transaction codes — zero stack trace reading required by developer.

5. Deployment Checklist

5.1 Pre-Deployment

1. Create BTP api plan service key in Instances and Subscriptions
2. Add CPI_OAUTH_CLIENT to Security Material (clientid from service key)
3. Add CPI_OAUTH_SECRET to Security Material (clientsecret from service key)
4. Generate Gmail App Password at myaccount.google.com/apppasswords
5. Add SMTP_MAIL to Security Material (16-digit app password)
6. Add GEMINI_API_KEY to Security Material (AIzaSy... from aistudio.google.com)
7. Replace YOUR-TENANT and REGION in Script 1 and Script 3

5.2 iFlow Configuration

8. Timer set to: Repeat=Minutes, Every=5
9. Router condition: ${property.hasAlerts} = 'true' on Alert Route
10. Router Route 2 marked as Default
11. Mail Adapter Connection: smtp.gmail.com:587, STARTTLS, Credential=SMTP_MAIL
12. Mail Adapter Processing: From, To, Subject fields filled

5.3 Post-Deployment Testing

13. Deploy DummyFailureGenerator with errorType=TIMEOUT — verify Auto-Healer retries
14. Deploy DummyFailureGenerator with errorType=ENRICH — verify email arrives
15. Check CPI trace logs show all 5 steps executing
16. Enable Trace log level on Auto-Healer for first week of production

6. Futuristic Approach — What Else We Can Try

The current Auto-Healer is version 1.0 — a reactive system that fixes failures after they happen. The future roadmap extends it into a predictive, self-learning, and business-aware intelligence layer for SAP integrations.

6.1 Short Term

REFORMAT Payload Auto-Fix

Currently REFORMAT errors only send alerts. The extended version asks Gemini to rewrite the payload with correct field names and resubmit automatically. For example, if a supplier sends invoice_number instead of inv_no, Gemini renames all fields and resubmits — zero human work.

- Gemini receives original payload + error text
- Returns corrected JSON with proper SAP field names
- Script 3 uses the fixedPayload to resubmit
- Handles: field renaming, date format conversion, currency code normalization, amount formatting, extra field stripping

Slack Webhook Notifications

Replace or supplement the Mail adapter with a Slack webhook call. Alerts arrive instantly in the team channel with color-coded severity, reducing email noise.

- Green message: auto-fixed successfully
- Yellow message: needs human attention
- Red message: critical business error escalated

Multi-Channel Alert Routing

Route different error types to different people. ENRICH errors go to the SAP master data team. ESCALATE auth errors go to the Basis team. REFORMAT errors go to the integration developer.

6.2 Medium Term — Next Quarter

Predictive Failure Detection

Instead of reacting to failures, analyze patterns to predict them before they happen. The Auto-Healer reviews the last 7 days of message statistics and uses Gemini to identify warning signs.

- High error rate on specific iFlow trending upward
- Processing time increasing — target system slowing down
- Specific partner always failing on Mondays — batch timing issue

- End-of-month volume spikes causing timeouts — scale proactively

Pattern Detected	Prediction	Action
Error rate 3x higher today vs last week	iFlow will fail tonight batch run	Alert team to investigate before batch
Processing time increasing by 20% daily	Target system degrading	Alert Basis team to check SAP performance
Same error every Friday at 6 PM	Partner sends bad data end of week	Auto-fix rule created for this partner
New partner first week — 40% error rate	Partner not correctly configured	Escalate to partner onboarding team

Self-Learning Error Patterns

The Auto-Healer stores every error and its resolution in a CPI Data Store. Over time it learns patterns specific to your landscape — which vendors always trigger ENRICH errors, which iFlows always timeout on Monday mornings.

- After 10 occurrences of same error type from same partner — create auto-fix rule
- After 5 consecutive RETRY failures from same target — escalate to Basis immediately
- After 3 REFORMAT errors from same supplier — alert partner team to fix their API

Integration Health Dashboard

A dedicated monitoring iFlow generates a daily HTML health report showing error trends, auto-fix rates, and top problematic iFlows. Posted to SharePoint or emailed every morning at 8 AM.

- Total failures per day chart (last 30 days)
- Auto-fix rate percentage — target: above 60%
- Top 5 most problematic iFlows
- Top 5 most common error types
- Partner scorecard — which trading partners send the most errors

6.3 Long Term — Next 6 Months

Natural Language Integration Monitoring

Allow non-technical team members to query the integration health using plain English via a Teams or Slack bot.

User Question	Auto-Healer Response
How many invoices failed today?	3 invoices failed — 2 auto-fixed, 1 needs vendor creation (XK01)
Why is the PO iFlow slow?	Average processing time 45s vs normal 8s. Target SAP system CPU at 92% since 10 AM
Which supplier sends the most errors?	ACME Corp — 23 errors this month, mostly wrong date format
Is the S/4HANA connection healthy?	Yes — 99.2% success rate last 24h. Last timeout was 3 days ago

AI-Powered Partner Onboarding Assistant

When a new trading partner sends their first message and it fails, the Auto-Healer generates a complete onboarding report showing exactly what needs to be configured — no developer investigation needed.

- Analyses the first 10 messages from a new partner
- Identifies all field mapping mismatches
- Generates a suggested mapping configuration
- Creates draft Groovy script for the partner-specific transformation
- Sends onboarding checklist to the partner team

Cross-System Correlation Analysis

Connect the Auto-Healer to SAP Solution Manager or Focused Run to correlate CPI integration errors with SAP system events — connecting the dots between integration failures and their root causes in the backend.

- CPI invoice posting failed at 2:15 AM
- SAP S/4HANA had planned maintenance 2:00-2:30 AM
- Auto-Healer correlates these and schedules retry for 2:35 AM automatically
- No alert sent — human-defined maintenance windows are respected

Generative Integration Documentation

Use Gemini to automatically generate and keep up-to-date documentation for every iFlow based on its behavior, common errors, and resolutions.

- iFlow description generated from code and naming
- Common errors and their solutions — learned from Auto-Healer history
- Partner-specific quirks documented automatically
- Runbook pages in Confluence updated weekly by the Auto-Healer

6.4 Vision — The Autonomous Integration Platform

The long-term vision is an integration platform that operates autonomously — where integration failures are an exception rather than a daily occurrence, and when they do happen, they are resolved before any human even notices.

Today (v1.0)	6 Months (v2.0)	12 Months (v3.0)
React to failures	Predict failures	Prevent failures
Fix RETRY automatically	Fix REFORMAT automatically	Fix ENRICH by creating SAP data
Alert with diagnosis	Alert with fix already applied	No alert — resolved silently
Monitor CPI only	Monitor CPI + SAP systems	Monitor full end-to-end business process
English error reports	Natural language chat interface	Voice-activated integration control

The AI Auto-Healer is not just a monitoring tool — it is the foundation of an intelligent, self-managing integration platform. Every error it learns from makes the next error less likely to need human attention.

7. Common Errors and Fixes

Error Message	Cause	Fix
unable to resolve class Message	Missing import statement	Add: import com.sap.gateway.ip.core.customdev.util.Message
No such property: secureStoreService	Wrong API for Script step	Use ITApiFactory.getApi(SecureStoreService.class, null)
[C.trim() not applicable	Password is char array not String	Use new String(credential.password).trim()
401 UNAUTHENTICATED	Wrong OAuth credentials	Check CPI_OAUTH_CLIENT and CPI_OAUTH_SECRET values
404 route does not exist	Wrong cpiHost URL	Use it-cpitrial03 URL not integrationsuite URL for API
500 ClassCastException Timestamp	Wrong datetime format in filter	Use SimpleDateFormat not Instant.now()
RejectedExecutionException	HTTP client created inside loop	Create CloseableHttpClient once outside loop
JsonException curly quotes	Gemini returned Unicode quotes	Add .replace('\u201C','"').replace('\u201D','"')
PatternSyntaxException near {	Unescaped curly braces in regex	Use \\{ and \\} to escape curly braces in Groovy regex
Router condition invalid format	String comparison with > operator	Use = 'true' not > 0 for string properties
503 Service Manager not bound	Mail adapter on trial tenant	Remove Mail adapter, use Slack webhook or check Object Store binding
HTML returned instead of JSON	OAuth token not accepted by API	Create new BTP service key with api plan not integration-flow plan

SAMPLE Alerts Result →

CPI Auto-Healer Alert — 1 issue(s) found

1 message

<achalpratapsingh1220@gmail.com> Wed, Jul 1, 2026 at 12:01 PM
To: thakurachalpratapsinghh@gmail.com

```
SAP CPI AI AUTO-HEALER REPORT
============================================================
Generated : 01-Jul-2026 06:31:49
============================================================

SUMMARY
------------------------------
  Issues requiring attention : 1
  Auto-fixed by AI         : 0

ISSUES REQUIRING YOUR ATTENTION
============================================================

ISSUE 1 of 1
------------------------------------------------------------
iFlow Name    : DummyFailureGenerator
Error Type    : ENRICH
Time          : /Date(1782887481623)/
Message ID    : AGpEtDmY_yLpdouD0xYlD1qYoCvR
AI Confidence : 75%

WHAT WENT WRONG
  javax.script.ScriptException: java.lang.Exception: java.lang.Exception: BAPI_PO_

ORIGINAL ERROR
  javax.script.ScriptException: java.lang.Exception: java.lang.Exception: BAPI_PO_CREATE failed. Vendor GLOBEX_CORP_999 does not
exist in company code 1000. Please create vendor using transaction XK01.@ line 19 in script1.groovy, cause: java.lang.Exception:
BAPI_PO_CREATE failed. Vendor GLOBEX_CORP_99

HOW TO FIX IT
  Step 1: create missing master data in SAP
  Step 2: 2: retry message from CPI Monitor.

SAP TRANSACTION
  Open SAP and run: XK01 / MM01 / KS01

QUICK REFERENCE
  Create vendor  : Transaction XK01
  Create material: Transaction MM01
  Create cost ctr: Transaction KS01
  Create plant   : Transaction OX10
  After creating master data:
  CPI Monitor - find Message ID above
  Click Retry to reprocess

------------------------------------------------------------

============================================================
SAP CPI AI Auto-Healer - Powered by Gemini AI
============================================================
```